The per-location Gaussian NLL term in zrg_cost_function_gaussian_nll

What this document covers

This note explains, line by line, the three-line core of the per-location helper inside zrg_cost_function_gaussian_nll (src/autotune_gp/cost.py), reproduced here for reference:

```
var_c = np.maximum(pv, var_floor)
term  = 0.5 * (pp - po) ** 2 / var_c + 0.5 * np.log(2 * np.pi * var_c)
return np.sum(weights * term) / np.sum(weights)   # or np.mean(term) if weights is None
```

po and pp are arrays of observed and predicted values at every individual location within one snapshot/block/variable group (e.g. the 18 zonal bands for one variable in the annual snapshot); pv is the GP's own predictive variance at those same locations, for whichever parameter vector x the optimizer is currently evaluating.

Line 1 — the variance floor

```
var_c = np.maximum(pv, var_floor)
```

pv comes straight out of gp.predict(x) — it is the multi-fidelity AR1 model's posterior predictive variance at this specific location, evaluated at this specific point in (normalized) parameter space. It is not a fixed number: it shrinks near training data (especially high-fidelity training points) and grows in regions the surrogate has to extrapolate into or lean on the low-fidelity blend for.

The floor exists purely as a numerical safety net, not a modeling choice. Two of the terms below divide by var_c or take its logarithm, and both blow up as var_c → 0:

- 1/var_c → ∞ (fast — this is the dangerous one)

- log(var_c) → -∞ (slower, but still unbounded)

A near-zero predictive variance is a real possibility here, not a hypothetical edge case: it happens right at training points, and can happen more broadly if one of the many independent per-feature AR1 models' fitted noise term collapses toward zero during optimization — a known pathology of GP regression on near-deterministic simulator output. Without the floor, a tiny residual landing next to a near-zero variance could produce an enormous or infinite cost value, which would destabilize the L-BFGS-B search (it expects smooth, well-behaved gradients, not occasional cliffs). var_floor defaults to 1e-6 in normalized (standardized) Y-space, where meaningful variances are O(1) — so in practice this floor is inert almost everywhere and only intervenes as an emergency clamp.

Line 2 — the per-location Gaussian negative-log-likelihood term

```
term = 0.5 * (pp - po) ** 2 / var_c + 0.5 * np.log(2 * np.pi * var_c)
```

This is exactly -log N(po | pp, var_c) — the negative log of the Gaussian probability density, evaluated at the observation po, under a normal distribution centered at the GP's prediction pp with variance var_c. It has two additive pieces, and both matter:

1. The fit term, 0.5*(pp-po)²/var_c. This is a squared residual, scaled by how much the GP trusts itself at that location. A location where the GP is confident (small var_c) turns even a modest miss into a large penalty; a location where the GP is uncertain (large var_c) discounts the same miss heavily. This is the mechanism, discussed earlier, that stops the optimizer from being rewarded for landing in a poorly-supported region of parameter space just because the surrogate's mean happens to look good there.

2. The normalization term, 0.5*log(2π·var_c) — the subject of the next section.

Both operations here are elementwise: every location keeps its own var_c, divides its own squared residual by it, and gets its own log term. Nothing is pooled or averaged yet — that happens only in line 3.

Line 3 — the weighted mean across locations

```
return np.sum(weights * term) / np.sum(weights)   # or np.mean(term) if weights is None
```

Only now, after every location already has its own complete NLL contribution, do the values get combined — as a weighted mean, not a sum, across the locations within one zonal/regional block. This is a deliberate choice: nearby zonal bands or regions in a climate field are spatially correlated, not independent observations, so summing would overstate how much genuine information a spatially-smooth pattern actually contains and would make the fit look more certain than it is. The optional weights argument (zonal_weights/regional_weights from the config) lets specific locations be emphasized or de-emphasized within that mean — e.g. for area-weighting — the same role these weights already played in the pre-existing MAE cost function.

Why the log(variance) term matters — the part worth dwelling on

Dropping the normalization term and using only 0.5*r²/var_c looks tempting — it's simpler, and it still "rewards" small residuals. But it creates an uncontrolled incentive: for any fixed residual r, this truncated score can be driven arbitrarily close to zero just by making var_c large, with no penalty for doing so. Concretely, with r = 1:

var_c	0.5·r²/var_c (no log term)	full NLL (with log term)
0.01	50.000	48.62
1	0.500	1.42
100	0.005	3.23

Without the log term, the "cost" strictly decreases as var_c grows — inflating the variance is always a free win, regardless of whether the fit is any good. That is exactly the overconfidence-relocation problem this whole feature exists to avoid: instead of the optimizer exploiting a region where the GP is silently overconfident, you'd have the cost function itself exploitable by any process (the trained GP's own variance estimate, or indirectly the optimizer's choice of where to search) that can inflate var_c.

With the log term included, the picture flips to what it should be: the total is smallest when var_c actually matches the true spread of the residual, and gets worse in *both* directions — too small a var_c is punished harshly (a confident-but-wrong prediction, var_c=0.01 above, costs 48.6), and too large a var_c is also punished, just more gently (an falsely-vague prediction, var_c=100, costs 3.2 rather than the 0.005 it would "cost" without the log term). This is not a coincidence — it is the defining property of a strictly proper scoring rule: if the residual really is drawn from N(0, σ²_true), then the expected value of this score, as a function of the assumed var_c, is minimized exactly at var_c = σ²_true and nowhere else. That can be shown directly by differentiating the expected score with respect to var_c and setting it to zero:

```
d/d(var_c) E[ r²/(2·var_c) + 0.5·log(2π·var_c) ] = -E[r²]/(2·var_c²) + 1/(2·var_c) = 0
  <=>  var_c = E[r²]
```

i.e. the score is minimized precisely when the assumed variance equals the true mean squared residual — not at zero, not at infinity. Removing the log term removes this fixed point entirely; the derivative becomes strictly negative for all var_c > 0, so there is no longer any interior minimum and the "score" is minimized only in the degenerate limit var_c → ∞.

For this pipeline specifically, the physical tuning parameters x don't set var_c directly — it comes from the trained GP — but x does determine *where in parameter space* you are, and the GP's variance genuinely does vary across that space (larger away from HF training data, or where the model leans more on the LF blend). Without the log term, the cost function would have a structural bias toward regions of high GP uncertainty, since those regions mechanically shrink the fit term regardless of whether the true fit is good — silently reintroducing the exact failure mode (the optimizer being rewarded for landing somewhere the surrogate can't actually vouch for) that motivated switching away from plain weighted MAE in the first place. The log term is what closes that loophole: a region of inflated uncertainty is never a free way to lower the cost, because uncertainty is priced into the score at exactly the rate it's discounting the residual by.

References

- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press. Freely available at gaussianprocess.org/gpml. Chapter 2 gives the same fit-term-plus-normalization structure for a GP's log marginal likelihood (-log p(y|X) = 0.5·yᵀK⁻¹y + 0.5·log|K| + (n/2)·log2π); the per-location term used here is that expression's diagonal (single-output, independent-locations) special case.

- Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359-378. The canonical reference for why a forecast's predicted mean and variance must be scored jointly, and what "proper" means for a scoring rule.

- Nix, D. A., & Weigend, A. S. (1994). Estimating the mean and variance of the target probability distribution. *Proceedings of the 1994 IEEE International Conference on Neural Networks*, 1, 55-60. One of the earliest papers to use this exact heteroscedastic Gaussian NLL as a training loss, with explicit discussion of the log-variance term's role in preventing variance collapse/inflation.

- Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems (NeurIPS) 30*. arXiv:1612.01474. A widely-cited, easy-to-find modern use of the same loss.